

Memoria Práctica 2 - Sistemas Informaticos II

Asignatura: Sistemas Informáticos II

Grupo: 2311

Pareja: Alejandro Pardo, Pablo Pollo

Fecha: 03/04/2026

Introducción

La presente memoria documenta el desarrollo de la Práctica 2 de la asignatura Sistemas Informáticos II, cuyo objetivo es transformar una aplicación Django monolítica (visaApp) en una arquitectura distribuida cliente-servidor. Para ello se han empleado dos mecanismos de comunicación fundamentales en sistemas distribuidos: llamadas a procedimiento remoto (RPC) mediante la biblioteca Modern RPC con protocolo XML-RPC, y colas de mensajes asíncronas mediante RabbitMQ con la librería Pika.

La metodología seguida ha consistido en dividir el trabajo en fases incrementales, completando y verificando cada una antes de avanzar a la siguiente. En primer lugar se creó y probó el backend RPC en local, exportando las funciones de acceso a la base de datos como procedimientos remotos. A continuación se construyó el frontend RPC, reescribiendo la capa de acceso a datos para que invoque procedimientos remotos en lugar de utilizar el ORM de Django. Una vez validados ambos componentes mediante tests automatizados, se desplegaron en máquinas virtuales independientes (VM2 para el backend, VM3 para el frontend y VM1 para PostgreSQL y RabbitMQ), verificando la comunicación entre las tres máquinas en cada paso. Finalmente se implementó un servicio de cancelación de pagos basado en colas de mensajes, demostrando la comunicación asíncrona y la persistencia de mensajes en RabbitMQ. Esta aproximación fase a fase, junto con verificaciones sistemáticas del código y del empaquetado final, ha permitido evitar los errores detectados en prácticas anteriores y completar la práctica con éxito.

Ejercicio 1 - Creación del Proyecto Backend RPC

Pasos realizados

Se ha creado el proyecto P1-rpc-backend partiendo de P1-base mediante los siguientes comandos:

1. Copia del proyecto base: `cp -r P1-base P1-rpc-backend`
2. Renombrado de la aplicación: `mv visaApp visaAppRPCBackend` seguido de `find ./ -type f -exec sed -i "s/visaApp/visaAppRPCBackend/g" {} ;`
3. Eliminación de ficheros innecesarios: `forms.py`, `views.py`, `templates/`, `tests_views.py`, `tests_models.py`

Capturas

CAPTURA 1.1: Resultado de `ls visaAppRPCBackend/` mostrando que los ficheros innecesarios han sido eliminados.

```
(venv) e462971@1-21-1-21:~/Descargas/si2_alumnos-main/P1-rpc-backend$ cd /home/alumnos/e462971/Descargas/si2/si2_alumnos-main/P1-rpc-backend
ls visaAppRPCBackend/
bash: cd: /home/alumnos/e462971/Descargas/si2/si2_alumnos-main/P1-rpc-backend: No existe el archivo o el directorio
admin.py  __init__.py  migrations  pagoDB.py  tests_rpc_backend.py
apps.py   management  models.py   __pycache__  urls.py
```

CAPTURA 1.2: Resultado de `grep -r "visaApp" --include="*.py" . | grep -v "visaAppRPC"` mostrando que el renombrado no dejó residuos.

```
(venv) e462971@1-21-1-21:~/Descargas/si2_alumnos-main/P1-rpc-backend$ grep -r "visaApp" --include="*.py" . | grep -v "visaAppRPC"
(venv) e462971@1-21-1-21:~/Descargas/si2_alumnos-main/P1-rpc-backend$
```

¿Por qué no será necesario hacer uso de los formularios Django y las plantillas en la aplicación servidor RPC?

El backend RPC no interactúa directamente con usuarios a través del navegador. Su única función es recibir llamadas RPC programáticas procedentes de otro servidor (el frontend) y devolver datos serializados.

Los formularios Django (`forms.py`) sirven para validar y procesar datos que provienen de formularios HTML enviados por el navegador. En el backend RPC no existe ningún formulario HTML, ya que los datos llegan como parámetros dentro de peticiones XML-RPC o JSON-RPC.

Las plantillas (`templates/`) son ficheros HTML que Django renderiza para generar las páginas web que ve el usuario. El backend RPC no genera páginas web; únicamente expone un endpoint POST (`/rpc/`) que recibe llamadas de procedimiento remoto y devuelve los resultados como datos serializados (XML o JSON). Por estas razones, ni los formularios ni las plantillas son necesarios en la aplicación servidor RPC.

Ejercicio 2 - Exportación de la Funcionalidad como Procedimientos Remotos

Pasos realizados

1. Configuración de settings.py: se añadió "modernrpc" a INSTALLED_APPS y se definió MODERNRPC_METHODS_MODULES apuntando a visaAppRPCBackend.pagoDB.
2. Reescritura de urls.py: se eliminaron todas las rutas web de P1-base y se dejó un único endpoint /rpc/ usando RPCEntryPoint de Modern RPC.
3. Modificación de pagoDB.py: se añadió el decorador @rpc_method a las 4 funciones y se modificaron registrar_pago y get_pagos_from_db para que devuelvan diccionarios usando model_to_dict.

Capturas

CAPTURA 2.1: Resultado de grep -i "modernrpc" visaSite/settings.py mostrando las 2 líneas de configuración.

```
• (venv) e462971@1-21-1-21:~/Descargas/si2_alumnos-main/P1-rpc-backend$ grep -i "modernrpc" visaSite/settings.py
    "modernrpc",
    MODERNRPC_METHODS_MODULES = [
• (venv) e462971@1-21-1-21:~/Descargas/si2_alumnos-main/P1-rpc-backend$
```

CAPTURA 2.2: Contenido completo de urls.py con cat visaAppRPCBackend/urls.py.

```
• (venv) e462971@1-21-1-21:~/Descargas/si2_alumnos-main/P1-rpc-backend$ cat visaAppRPCBackend/urls.py
"""
URL configuration for PagoProj project.

The `urlpatterns` list routes URLs to views. For more information please see:
    https://docs.djangoproject.com/en/4.2/topics/http/urls/
Examples:
Function views
    1. Add an import:  from my_app import views
    2. Add a URL to urlpatterns:  path('', views.home, name='home')
Class-based views
    1. Add an import:  from other_app.views import Home
    2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
Including another URLconf
    1. Import the include() function: from django.urls import include, path
    2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
"""
from django.urls import path
from modernrpc.views import RPCEntryPoint

urlpatterns = [
    path("rpc/", RPCEntryPoint.as_view(), name="rpc")
]
```

CAPTURA 2.3: Resultado de grep -n "@rpc_method|model_to_dict" visaAppRPCBackend/pagoDB.py mostrando los decoradores y usos de model_to_dict.

```
• (venv) e462971@1-21-1-21:~/Descargas/si2_alumnos-main/P1-rpc-backend$ grep -n "@rpc_method\\|model_to_dict" visaAppRPCBackend/pagoDB.py
10:from django.forms.models import model_to_dict
13:@rpc_method
26:@rpc_method
39:    pago_a_devolver = model_to_dict(pago)
44:@rpc_method
58:@rpc_method
67:    pago_dict = model_to_dict(pago)
```

¿Por qué es necesario hacer uso del método `model_to_dict` tras registrar un pago?

El protocolo XML-RPC solo puede transmitir tipos básicos de Python: cadenas de texto, enteros, números de punto flotante, booleanos, listas y diccionarios. Un objeto Pago de Django es una instancia del ORM que no pertenece a ninguno de estos tipos básicos. Si se intentase devolver directamente el objeto Pago a través de una llamada RPC, la serialización XML-RPC fallaría porque no sabe cómo convertir un objeto Django a XML.

El método `model_to_dict` resuelve este problema convirtiendo el objeto del modelo Django en un diccionario Python estándar, que sí es un tipo serializable por XML-RPC y puede transmitirse correctamente por la red.

Ejercicio 3 - Prueba del Backend RPC

Pasos realizados

1. Configuración del fichero env con la URL de la base de datos.
2. Ejecución de migraciones: `python manage.py makemigrations` y `python manage.py migrate`.
3. Población de la base de datos: `python manage.py populate`.
4. Arranque del servidor: `python manage.py runserver 8001`.
5. Verificación en el navegador accediendo a `http://127.0.0.1:8001/visaAppRPCBackend/rpc`.
6. Ejecución de los tests proporcionados.

Capturas

CAPTURA 3.1: Resultado de `python manage.py migrate` mostrando las migraciones aplicadas sin errores.

CAPTURA 3.2: Resultado de `python manage.py populate` mostrando los mensajes de éxito.

```

• (venv) e462971@1-21-1-21:~/Descargas/si2_alumnos-main/P1-rpc-backend$ python manage.py makemigrations
python manage.py migrate
python manage.py populate
No changes detected
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions, visaAppRPCBackend
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
  Applying visaAppRPCBackend.0001_initial... OK
Database cleaned successfully
Tarjeta objects created successfully

```

CAPTURA 3.3: Navegador mostrando el error HTTP 405 al acceder a la URL del endpoint RPC. Este error es el comportamiento esperado, ya que el endpoint sólo acepta peticiones POST (con payloads XML RPC/JSON-RPC) y el navegador realiza peticiones GET.

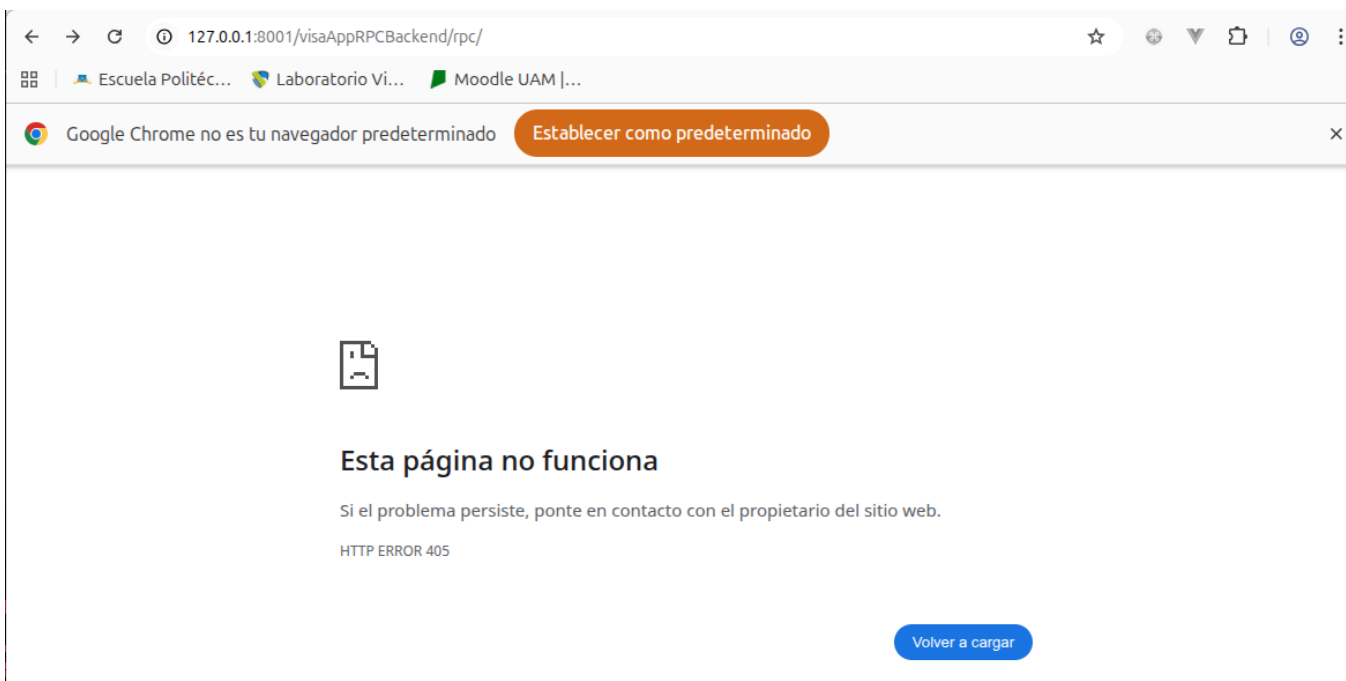
```

• (venv) e462971@1-21-1-21:~/Descargas/si2_alumnos-main/P1-rpc-backend$ python manage.py runserver 8001
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
March 02, 2026 - 11:44:55
Django version 5.2.2, using settings 'visaSite.settings'
Starting development server at http://127.0.0.1:8001/
Quit the server with CONTROL-C.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/5.2/howto/deployment/
[02/Mar/2026 11:45:05] "GET /visaAppRPCBackend/rpc HTTP/1.1" 301 0
Method Not Allowed (GET): /visaAppRPCBackend/rpc/
Method Not Allowed: /visaAppRPCBackend/rpc/
[02/Mar/2026 11:45:05] "GET /visaAppRPCBackend/rpc/ HTTP/1.1" 405 0

```



CAPTURA 3.4: Resultado de `python manage.py test visaAppRPCBackend.tests_rpc_backend` mostrando Ran 4 tests ... OK.

```
• (venv) e462971@1-21-1-21:~/Descargas/si2_alumnos-main/P1-rpc-backend$ python manage.py test visaAppRPCBackend.tests_rpc_backend
Found 4 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
....
-----
Ran 4 tests in 0.049s

OK
Destroying test database for alias 'default'...
```

Ejercicio 4 - Despliegue del Backend RPC en la VM

Pasos realizados

1. Copia del proyecto a la VM2 mediante `scp -r -P 22022 P1-rpc-backend si2@localhost:~/`.
2. Configuración del fichero `env` en la VM2 con `DATABASE_SERVER_URL` apuntando a la VM1 (10.0.2.2:15432).
3. Instalación de dependencias: `pip install -r requirements.txt`.
4. Ejecución de migraciones y población de la base de datos.
5. Arranque del servidor con `python manage.py runserver 0.0.0.0:8000`.
6. Verificación desde el PC del laboratorio accediendo a `localhost:28000/visaAppRPCBackend/rpc`.

Esquema de despliegue

PC Laboratorio --> localhost:28000 --> VM2 (Backend RPC)

|

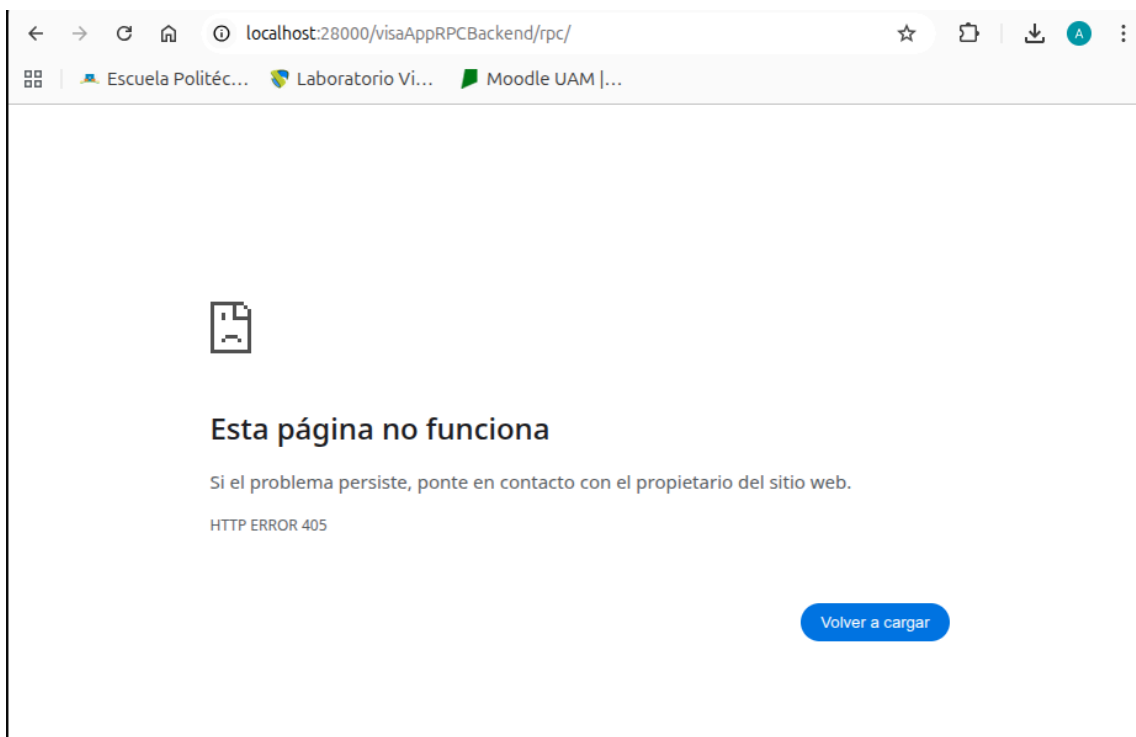
10.0.2.2:15432

v

VM1 (PostgreSQL)

Capturas

CAPTURA 4.1: Navegador mostrando el error HTTP 405 al acceder a `localhost:28000/visaAppRPCBackend/rpc/`, demostrando que el backend está desplegado y accesible desde fuera de la VM.



Ejercicio 5 - Creación del Proyecto Frontend RPC

Pasos realizados

1. Copia del proyecto base: `cp -r P1-base P1-rpc-frontend`.
2. Renombrado de la aplicación a `visaAppRPCFrontend`.
3. Eliminación de ficheros innecesarios: `models.py`, `migrations/`, `management/`, `tests_models.py`, `tests_views.py`.
4. Configuración del fichero `env` añadiendo la variable `RPCAPIBASEURL`.
5. Modificación de `settings.py` para leer `RPCAPIBASEURL` con `os.environ.get("RPCAPIBASEURL")`.

Capturas

CAPTURA 5.1: Resultado de `ls visaAppRPCFrontend/` mostrando la estructura limpia.

```
e462971@6B-12-18-12:~/Descargas/P2_SI2/si2_alumnos-main/P1-rpc-frontend$ ls visaAppRPCFrontend/
admin.py  forms.py      pagoDB.py  tests_views.py  views.py
apps.py   __init__.py  templates  urls.py
e462971@6B-12-18-12:~/Descargas/P2_SI2/si2_alumnos-main/P1-rpc-frontend$
```

CAPTURA 5.2: Resultado de `grep -r "visaApp" --include="*.py" . | grep -v "visaAppRPC"` mostrando que no hay residuos del renombrado.

```
alejandro@alejandro-X99:~/Descargas/P2SI2/P2SI2/P2_SI2/si2_alumnos-main/P1-rpc-Frontend$ grep -r "visaApp" --include="*.py" . | grep -v "visaAppRPC" | grep -v "venv"
```

CAPTURA 5.3: Resultado de `grep "RPCAPIBASEURL" env` mostrando la variable configurada.

```
alejandro@alejandro-X99:~/Descargas/P2SI2/P2SI2/P2_SI2/si2_alumnos-main/P1-rpc-frontend$ grep "RPCAPIBASEURL" env
RPCAPIBASEURL=http://localhost:28000/visaAppRPCBackend/rpc/
```

CAPTURA 5.4: Resultado de grep "RPCAPIBASEURL" visaSite/settings.py mostrando que settings.py lee la variable.

```
alejandro@alejandro-X99:~/Descargas/P2SI2/P2SI2/P2_SI2/si2_alumnos-main/P1-rpc-frontend$ grep "RPCAPIBASEURL" visaSite/settings.py
RPCAPIBASEURL = os.environ.get("RPCAPIBASEURL")
```

¿Por qué no será necesario hacer uso de los modelos de datos Django en la aplicación cliente RPC?

Los modelos Django definen la estructura de las tablas en la base de datos y permiten interactuar con ella a través del ORM. En la arquitectura distribuida implementada, el frontend RPC no accede directamente a ninguna base de datos; toda la interacción con los datos se realiza invocando procedimientos remotos en el backend RPC mediante el protocolo XML-RPC.

Cuando el frontend necesita verificar una tarjeta o registrar un pago, envía una llamada RPC al backend, que es quien ejecuta las operaciones sobre la base de datos usando sus propios modelos. Los datos viajan entre frontend y backend como diccionarios Python estándar (serializados en XML), no como objetos del ORM de Django. Por lo tanto, los modelos de datos no tienen ninguna utilidad en el frontend RPC y su presencia solo generaría dependencias innecesarias (como la necesidad de configurar una base de datos local y ejecutar migraciones).

Ejercicio 6 - Codificación del Frontend RPC

Pasos realizados

Se ha reescrito completamente el fichero pagoDB.py del frontend. En lugar de acceder a la base de datos mediante el ORM de Django, ahora cada función crea un proxy XML-RPC usando ServerProxy de la librería estándar xmlrpc.client e invoca el procedimiento remoto correspondiente del backend.

Las 4 funciones implementadas son:

verificar_tarjeta(tarjeta_data): invoca proxy.verificar_tarjeta() y devuelve True/False.

registrar_pago(pago_dict): invoca proxy.registrar_pago() y devuelve un diccionario con los datos del pago registrado.

eliminar_pago(idPago): invoca proxy.eliminar_pago() y devuelve True/False.

get_pagos_from_db(idComercio): invoca proxy.get_pagos_from_db() y devuelve una lista de diccionarios.

Capturas

CAPTURA 6.1: Contenido completo de pagoDB.py con cat visaAppRPCFrontend/pagoDB.py.


```

alejandro@alejandro-X99:~/Descargas/P25I2/P25I2/P2_5I2/si2_alumnos-main/P1-rpc-frontend$ cat visaAppRPCFrontend/pagoDB.py
"""Interface with the RPC backend"""
from django.conf import settings
from xmlrpc.client import ServerProxy

def verificar_tarjeta(tarjeta_data):
    """Verifica si una tarjeta esta registrada en la BD remota.
    :param tarjeta_data: diccionario con los datos de la tarjeta
                        (numero, nombre, fechaCaducidad, codigoAutorizacion)
    :return: True si la tarjeta existe, False en caso contrario
    """
    with ServerProxy(settings.RPCAPIBASEURL) as proxy:
        return proxy.verificar_tarjeta(tarjeta_data)

def registrar_pago(pago_dict):
    """Registra un pago invocando el procedimiento remoto del backend.
    :param pago_dict: diccionario con datos del pago
                    (idComercio, idTransaccion, importe, tarjeta_id)
    :return: diccionario con los datos del pago registrado
            (id, idComercio, idTransaccion, importe, marcaTiempo,
            codigoRespuesta, tarjeta). None si hubo error.
    """
    with ServerProxy(settings.RPCAPIBASEURL) as proxy:
        return proxy.registrar_pago(pago_dict)

def eliminar_pago(idPago):
    """Elimina un pago invocando el procedimiento remoto del backend.
    :param idPago: ID (entero) del pago a eliminar
    :return: True si se elimino correctamente, False si no existe
    """
    with ServerProxy(settings.RPCAPIBASEURL) as proxy:
        return proxy.eliminar_pago(idPago)

def get_pagos_from_db(idComercio):
    """Obtiene los pagos de un comercio invocando el procedimiento remoto.
    :param idComercio: ID del comercio a consultar
    :return: lista de diccionarios con los datos de cada pago
            (id, idComercio, idTransaccion, importe, marcaTiempo,
            codigoRespuesta, tarjeta)
    """
    with ServerProxy(settings.RPCAPIBASEURL) as proxy:
        return proxy.get_pagos_from_db(idComercio)

```

CAPTURA 6.2: Resultado de grep "objects" visaAppRPCFrontend/pagoDB.py mostrando salida vacía (no queda ningún acceso al ORM).

```

alejandro@alejandro-X99:~/Descargas/P25I2/P25I2/P2_5I2/si2_alumnos-main/P1-rpc-frontend$ grep "objects" visaAppRPCFrontend/pagoDB.py

```

¿Qué tipo de binding realiza el frontend RPC codificado?

El frontend RPC utiliza un binding estático (también conocido como static binding o fixed binding). La dirección del servidor RPC se define en el fichero de configuración env mediante la variable RPCAPIBASEURL y se carga al arrancar la aplicación a través de settings.RPCAPIBASEURL.

Esto significa que la ubicación del servidor backend está fijada en tiempo de configuración, no se descubre dinámicamente en tiempo de ejecución. El frontend no consulta ningún servicio de nombres ni directorio de servicios para localizar al backend; simplemente utiliza la URL que tiene configurada de antemano. Si el backend cambiase de dirección, habría que modificar manualmente el fichero env y reiniciar el frontend.

Ejercicio 7 - Prueba del Frontend RPC

Pasos realizados

1. Arranque del frontend con `python manage.py runserver 8001`.
2. Verificación manual en el navegador accediendo a <http://127.0.0.1:8001/visaAppRPCFrontend/>.
3. Prueba de registro de pago a través del flujo normal (tarjeta -> pago).
4. Prueba de las 3 operaciones desde testbd: registrar pago, listar pagos y borrar pago.
- 5.

Ejecución de los tests proporcionados.

```
(venv) e462971@6B-12-18-12:~/Descargas/P2_SI2/si2_alumnos-main/P1-rpc-frontend$ python manage.py runserver 8001
Watching for file changes with StatReloader
Performing system checks...

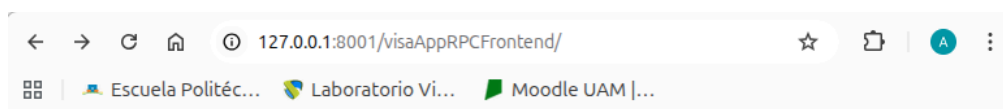
System check identified no issues (0 silenced).
March 03, 2026 - 16:24:01
Django version 5.2.2, using settings 'visaSite.settings'
Starting development server at http://127.0.0.1:8001/
Quit the server with CONTROL-C.

WARNING: This is a development server. Do not use it in a production setting. Use a production WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/5.2/howto/deployment/

[03/Mar/2026 16:24:38] "GET /visaAppRPCFrontend/ HTTP/1.1" 200 1115
-pract-modelo-memoria.pdf |.ico
[03/Mar/2026 16:24:38] "GET /favicon.ico HTTP/1.1" 404 2372
```

Capturas

CAPTURA 7.1: Navegador mostrando la página "Pago Registrado con Éxito" tras registrar un pago desde el flujo normal.



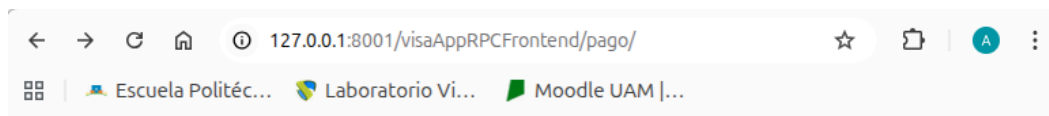
Introduzca la Información de la Tarjeta de la Persona que Paga (visaSite)

Número de Tarjeta:

Nombre:

Fecha de Caducidad:

Código de Autorización:



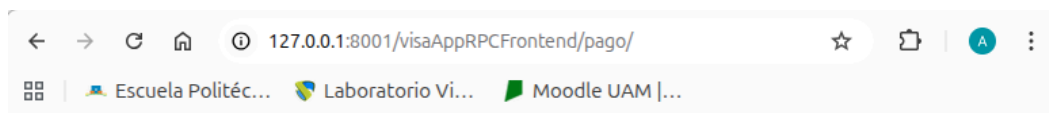
Registro de Pago (visaSite)

Introduzca los datos del nuevo pago a registrar:

ID Comercio:

ID Transaccion:

Importe:



Pago Registrado con Éxito (visaSite)

Id: 1

Codigo Respuesta: 000

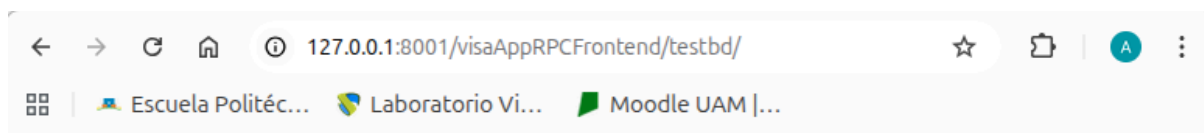
Marca Tiempo : 2026-03-03 16:32:46.123555+00:00

Id Comercio: COM001

Id Transaccion: TR001

Importe: 50.0

CAPTURA 7.2: Navegador mostrando la página de éxito tras registrar un pago desde testbd.



Test Base de Datos: Registro de Pago (visaSite)

Introduzca los datos del nuevo pago a registrar:

ID Comercio:

ID Transaccion:

Importe:

Número de Tarjeta:

Nombre:

Fecha de Caducidad:

Código de Autorización:

Test Base de Datos: Borrado de Pago

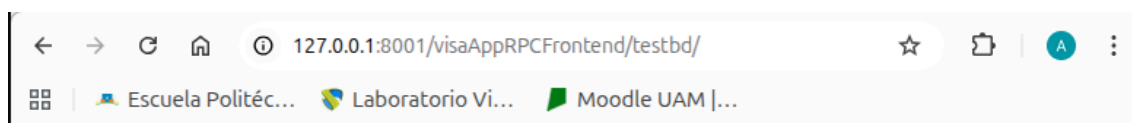
Introduzca el ID del Pago a Borrar:

ID del Pago:

Test Base de Datos: Listado de Pagos

Introduzca el ID del Comercio:

ID del Comercio:



Pago Registrado con Éxito (visaSite)

Id: 2

Codigo Respuesta: 000

Marca Tiempo : 2026-03-03 16:34:57.916635+00:00

Id Comercio: COM002

Id Transaccion: TR002

Importe: 75.0

CAPTURA 7.3: Navegador mostrando la tabla de pagos al listar pagos por idComercio.

Test Base de Datos: Registro de Pago (visaSite)

Introduzca los datos del nuevo pago a registrar:

ID Comercio:

ID Transaccion:

Importe:

Número de Tarjeta:

Nombre:

Fecha de Caducidad:

Código de Autorización:

Test Base de Datos: Borrado de Pago

Introduzca el ID del Pago a Borrar:

ID del Pago:

Test Base de Datos: Listado de Pagos

Introduzca el ID del Comercio:

ID del Comercio:

Pagos Registrados (visaSite)

id	IdComercio	IdTransaccion	Importe	Marca Tiempo	Codigo Respuesta
2	COM002	TR002	75.0	2026-03-03 16:34:57.916635+00:00	000

CAPTURA 7.4: Navegador mostrando el mensaje "Pago eliminado correctamente" tras borrar un pago.

← → ↻ 🏠 ⓘ 127.0.0.1:8001/visaAppRPCFrontend/testbd/ ☆ 📁 A ⋮

📦 | 🇸🇻 Escuela Politéc... 🇸🇻 Laboratorio Vi... 🇸🇻 Moodle UAM |...

Test Base de Datos: Registro de Pago (visaSite)

Introduzca los datos del nuevo pago a registrar:

ID Comercio:

ID Transaccion:

Importe:

Número de Tarjeta:

Nombre:

Fecha de Caducidad:

Código de Autorización:

Test Base de Datos: Borrado de Pago

Introduzca el ID del Pago a Borrar:

ID del Pago:

Test Base de Datos: Listado de Pagos

Introduzca el ID del Comercio:

ID del Comercio:

← → ↻ 🏠 ⓘ 127.0.0.1:8001/visaAppRPCFrontend/testbd/delpago/ ☆ 📁 A ⋮

📦 | 🇸🇻 Escuela Politéc... 🇸🇻 Laboratorio Vi... 🇸🇻 Moodle UAM |...

¡Pago eliminado correctamente!

CAPTURA 7.5: Resultado de python manage.py test mostrando Ran 8 tests...OK

```
(venv) e462971@68-12-18-12:~/Descargas/P2_SI2/si2_alumnos-main/P1-rpc-frontend$ python manage.py test
Found 8 test(s).
Creating test database for alias 'default'...
System check identified no issues (0 silenced).
.....
-----
Ran 8 tests in 0.269s

OK
Destroying test database for alias 'default'...
```

Ejercicio 8 - Despliegue del Frontend RPC en la VM

Pasos realizados

1. Copia del proyecto a la VM3 mediante `scp -r -P 32022 P1-rpc-frontend si2@localhost:~/`.
2. Configuración del fichero `env` en la VM3 con `RPCAPIBASEURL` apuntando al backend en la VM2 (<http://10.0.2.2:28000/visaAppRPCBackend/rpc/>).
3. Instalación de dependencias.
4. Arranque del servidor con `python manage.py runserver 0.0.0.0:8000`.
5. Prueba completa desde el navegador del PC del laboratorio.

Esquema de despliegue

Navegador (PC Laboratorio)

|

v

VM3:38000 -> Frontend RPC (vistas, formularios, templates)

|

| XML-RPC

v

VM2:28000 -> Backend RPC (pagoDB con @rpc_method, acceso a BD)

|

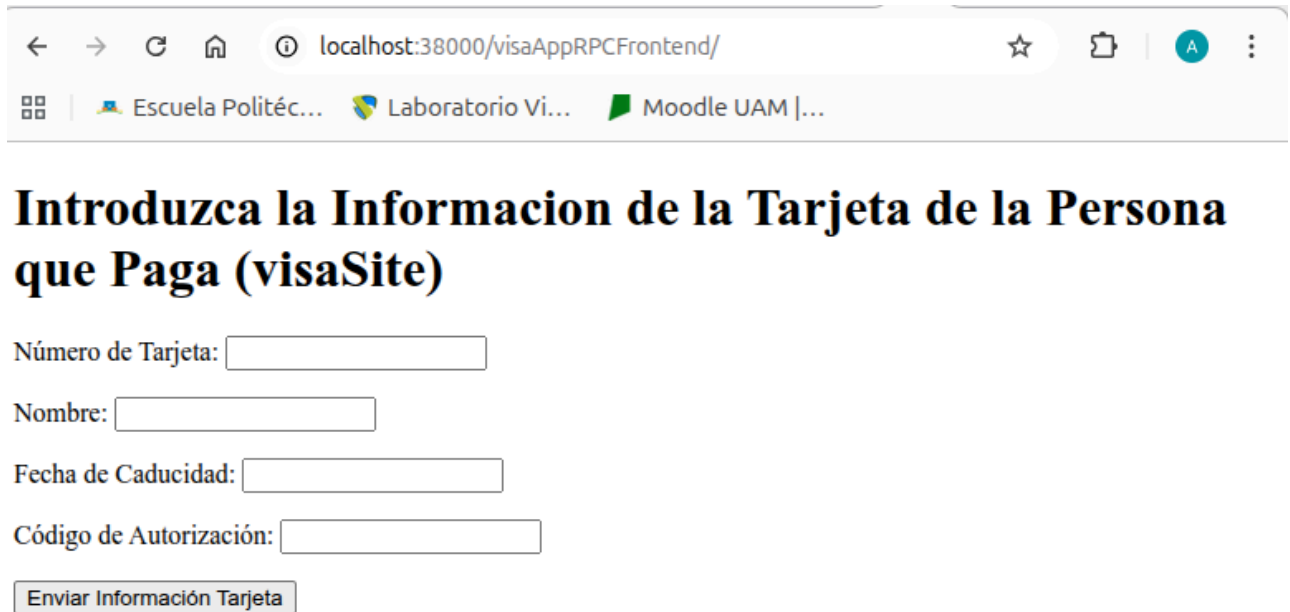
| SQL

v

VM1:15432 -> PostgreSQL (tablas tarjeta, pago)

Capturas

CAPTURA 8.1: Navegador mostrando el formulario de tarjeta accediendo a localhost:38000/visaAppRPCFrontend/.



The screenshot shows a web browser window with the address bar displaying 'localhost:38000/visaAppRPCFrontend/'. The browser's address bar also shows several tabs: 'Escuela Politéc...', 'Laboratorio Vi...', and 'Moodle UAM |...'. The main content area of the browser displays a form titled 'Introduzca la Informacion de la Tarjeta de la Persona que Paga (visaSite)'. The form contains four input fields: 'Número de Tarjeta:', 'Nombre:', 'Fecha de Caducidad:', and 'Código de Autorización:'. Below these fields is a button labeled 'Enviar Información Tarjeta'.

Introduzca la Informacion de la Tarjeta de la Persona que Paga (visaSite)

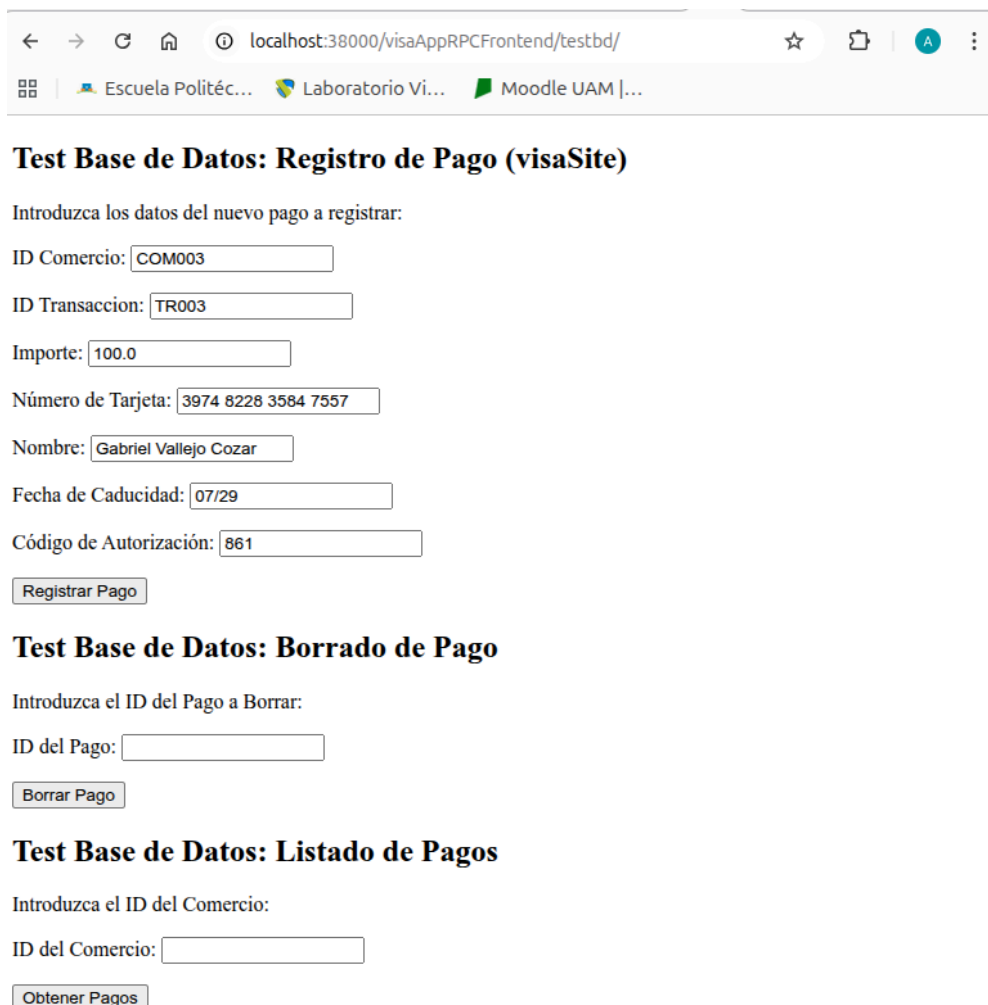
Número de Tarjeta:

Nombre:

Fecha de Caducidad:

Código de Autorización:

CAPTURA 8.2: Navegador mostrando la página de éxito al registrar un pago desde la VM3.



The screenshot shows a web browser window with the address bar displaying 'localhost:38000/visaAppRPCFrontend/testbd/'. The browser's address bar also shows several tabs: 'Escuela Politéc...', 'Laboratorio Vi...', and 'Moodle UAM |...'. The main content area of the browser displays a form titled 'Test Base de Datos: Registro de Pago (visaSite)'. The form contains several input fields: 'ID Comercio:', 'ID Transaccion:', 'Importe:', 'Número de Tarjeta:', 'Nombre:', 'Fecha de Caducidad:', and 'Código de Autorización:'. Below these fields is a button labeled 'Registrar Pago'.

Test Base de Datos: Registro de Pago (visaSite)

Introduzca los datos del nuevo pago a registrar:

ID Comercio:

ID Transaccion:

Importe:

Número de Tarjeta:

Nombre:

Fecha de Caducidad:

Código de Autorización:

Test Base de Datos: Borrado de Pago

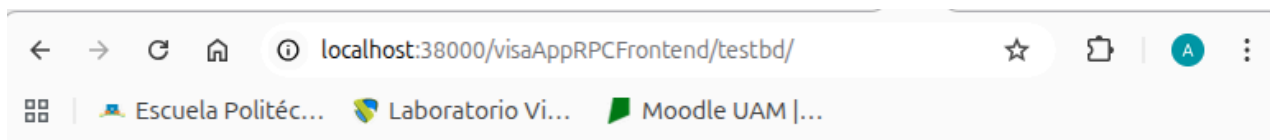
Introduzca el ID del Pago a Borrar:

ID del Pago:

Test Base de Datos: Listado de Pagos

Introduzca el ID del Comercio:

ID del Comercio:



Pago Registrado con Éxito (visaSite)

Id: 9

Codigo Respuesta: 000

Marca Tiempo : 2026-03-03 16:46:02.902161+00:00

Id Comercio: COM003

Id Transaccion: TR003

Importe: 100.0

CAPTURA 8.3: Navegador mostrando la tabla de pagos listados desde la VM3.

Test Base de Datos: Registro de Pago (visaSite)

Introduzca los datos del nuevo pago a registrar:

ID Comercio:

ID Transaccion:

Importe:

Número de Tarjeta:

Nombre:

Fecha de Caducidad:

Código de Autorización:

Test Base de Datos: Borrado de Pago

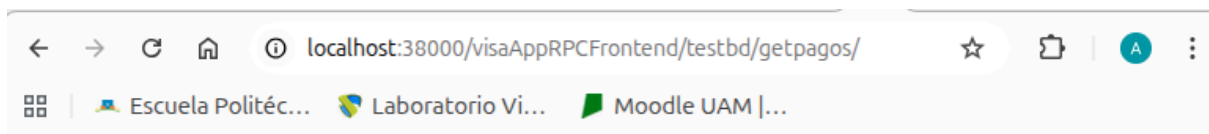
Introduzca el ID del Pago a Borrar:

ID del Pago:

Test Base de Datos: Listado de Pagos

Introduzca el ID del Comercio:

ID del Comercio:



Pagos Registrados (visaSite)

id	IdComercio	IdTransaccion	Importe	Marca Tiempo	Codigo Respuesta
9	COM003	TR003	100.0	2026-03-03 16:46:02.902161+00:00	000

CAPTURA 8.4: Navegador mostrando el mensaje de pago eliminado correctamente desde la VM3.

A screenshot of a web browser window showing a form titled 'Test Base de Datos: Registro de Pago (visaSite)'. The form contains several input fields for registration data: 'ID Comercio' (COM003), 'ID Transaccion' (TR003), 'Importe' (100,0), 'Número de Tarjeta' (3974 8228 3584 7557), 'Nombre' (Gabriel Vallejo Cozar), 'Fecha de Caducidad' (07/29), and 'Código de Autorización' (861). Below the fields is a 'Registrar Pago' button. The browser's address bar shows 'localhost:38000/visaAppRPCFrontend/testbd/'.

Test Base de Datos: Registro de Pago (visaSite)

Introduzca los datos del nuevo pago a registrar:

ID Comercio:

ID Transaccion:

Importe:

Número de Tarjeta:

Nombre:

Fecha de Caducidad:

Código de Autorización:

Test Base de Datos: Borrado de Pago

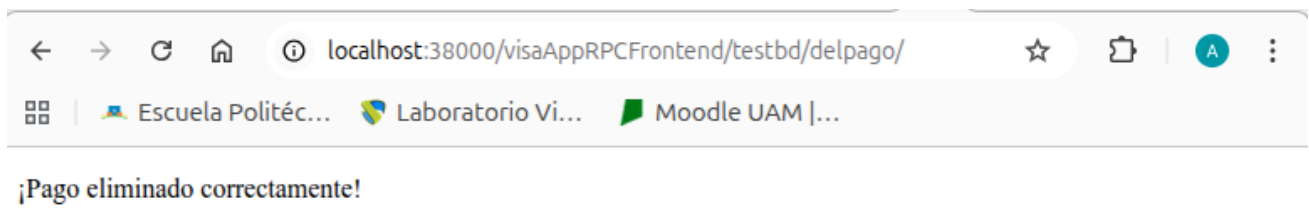
Introduzca el ID del Pago a Borrar:

ID del Pago:

Test Base de Datos: Listado de Pagos

Introduzca el ID del Comercio:

ID del Comercio:



Ejercicio 9 - Servicio de Cancelación de Pago

Descripción

Se ha creado el fichero `server_mq.py` en `P1-rpc-backend/visaAppRPCBackend/`. Este servidor se conecta al broker RabbitMQ mediante la librería Pika, declara una cola llamada `pago_cancelacion` y se queda esperando mensajes. Cuando recibe un mensaje con el ID de un pago, busca dicho pago en la base de datos y modifica su `codigoRespuesta` a '111' (cancelado).

El servidor implementa los siguientes pasos:

1. Conexión a RabbitMQ usando `pika.BlockingConnection` con credenciales `alumnomq/alumnomq`.
2. Declaración de la cola `pago_cancelacion` mediante `channel.queue_declare()`.
3. Función de callback que recibe el mensaje, decodifica el ID del pago, lo busca con `Pago.objects.get()` y cambia `codigoRespuesta` a '111'.
4. Registro del callback con `channel.basic_consume()` y comienzo del consumo con `channel.start_consuming()`.

Capturas

CAPTURA 9.1: Código completo de `server_mq.py` con `cat visaAppRPCBackend/server_mq.py`.

```

    ('(CodigoRespuesta= 111)')
(venv) e462971@6B-12-18-12:~/Descargas/P2_SI2/si2_alumnos-main/P1-rpc-backend$ cat visaAppRPCBackend/server_mq.py
# Uses rabbitMQ as the server

import os
import sys
import django
import pika

BASE_DIR = os.path.dirname(
    os.path.dirname(os.path.abspath(__file__)))
sys.path.append(BASE_DIR)

os.environ.setdefault('DJANGO_SETTINGS_MODULE',
                      'visaSite.settings')
django.setup()

from visaAppRPCBackend.models import Tarjeta, Pago

def main():

    if len(sys.argv) != 3:
        print("Debe indicar el host y el puerto")
        exit()

    hostname = sys.argv[1]
    port = sys.argv[2]

    # 1. Crear conexion con RabbitMQ
    credentials = pika.PlainCredentials('alumnomq', 'alumnomq')
    parameters = pika.ConnectionParameters(
        host=hostname,
        port=int(port),
        credentials=credentials
    )
    connection = pika.BlockingConnection(parameters)
    channel = connection.channel()

    # 2. Declarar la cola de mensajes
    channel.queue_declare(queue='pago_cancelacion')

    # 3. Definir la funcion de callback
    def callback(ch, method, properties, body):
        id_pago = body.decode()
        print(f"[x] Recibido mensaje: cancelar pago con ID={id_pago}")

```

Ejercicio 10 - Cliente de Cancelación de Pago

Descripción

Se ha creado el fichero client_mq.py en P1-rpc-frontend/cliente_mom/. Este cliente se conecta al broker RabbitMQ, declara la cola pago_cancelacion y publica un mensaje con el ID del pago a cancelar. El mensaje queda almacenado en la cola hasta que el servidor lo procese.

El cliente implementa los siguientes pasos:

1. Conexión a RabbitMQ usando `pika.BlockingConnection` con credenciales `alumnomq/alumnomq`.
2. Declaración de la cola `pago_cancelacion` (si no existe, se crea automáticamente).
3. Publicación del mensaje con `channel.basic_publish()` usando `routing_key='pago_cancelacion'` y el ID del pago como cuerpo del mensaje.
4. Cierre de la conexión con `connection.close()`.

Capturas

CAPTURA 10.1: Código completo de `client_mq.py` con `cat cliente_mom/client_mq.py`.

```
• (venv) e462971@6B-12-18-12:~/Descargas/P2_SI2/si2_alumnos-main/P1-rpc-frontend$ cat cliente_mom/client_mq.py
import pika
import sys

def cancelar_pago(hostname, port, id_pago):
    """Envia un mensaje a la cola de RabbitMQ para cancelar un pago.
    :param hostname: IP/hostname del servidor RabbitMQ
    :param port: puerto del servidor RabbitMQ
    :param id_pago: ID del pago a cancelar
    """
    try:
        # 1. Crear conexion con RabbitMQ
        credentials = pika.PlainCredentials('alumnomq', 'alumnomq')
        parameters = pika.ConnectionParameters(
            host=hostname,
            port=int(port),
            credentials=credentials
        )
        connection = pika.BlockingConnection(parameters)
    except Exception as e:
        print(f"Error al conectar al host remoto: {e}")
        exit()

    channel = connection.channel()

    # 2. Declarar la cola (si no existe, se crea)
    channel.queue_declare(queue='pago_cancelacion')

    # 3. Publicar el mensaje con el ID del pago
    channel.basic_publish(
        exchange='',
        routing_key='pago_cancelacion',
        body=str(id_pago)
    )

    print(f"[x] Enviado mensaje: cancelar pago con ID={id_pago}")

    # 4. Cerrar la conexion
    connection.close()

def main():
    if len(sys.argv) != 4:
        print("Debe indicar el host, el numero de puerto, "
              "y el ID del pago a cancelar como un argumento.")
        exit()
```

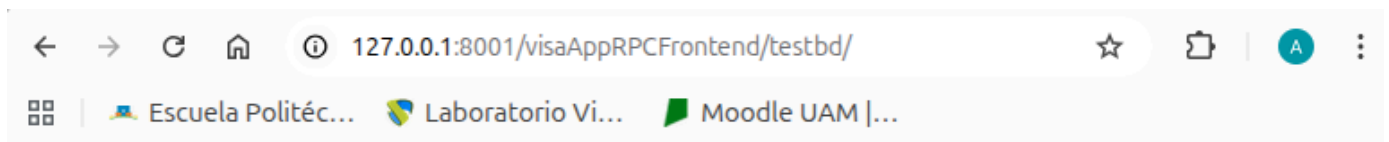
Ejercicio 11 - Ejecución de la Cancelación de Pago

Pasos realizados

1. Se registró un pago desde el frontend RPC con codigoRespuesta = 000.
2. Se ejecutó el cliente de cancelación desde el PC del laboratorio: `python client_mq.py localhost 15672 12`.
3. Se verificó que el mensaje se almacenó en la cola de RabbitMQ ejecutando `sudo rabbitmqctl list_queues` en la VM1.
4. Se arrancó el servidor de cancelación en la VM2: `python server_mq.py 10.0.2.2 15672`.
5. El servidor procesó el mensaje y canceló el pago (codigoRespuesta cambiado a '111').
6. Se verificó la cancelación listando los pagos desde el frontend RPC.
7. Se comprobó que la cola quedó vacía ejecutando nuevamente `sudo rabbitmqctl list_queues`.

Capturas

CAPTURA 11.1: Página de éxito al registrar el pago con codigoRespuesta = 000.



Pago Registrado con Éxito (visaSite)

Id: 10

Codigo Respuesta: 000

Marca Tiempo : 2026-03-03 16:57:28.968399+00:00

Id Comercio: CANCEL001

Id Transaccion: TRCANCEL01

Importe: 100.0

Pagos Registrados (visaSite)

id	IdComercio	IdTransaccion	Importe	Marca Tiempo	Codigo Respuesta
10	CANCEL001	TRCANCEL01	100.0	2026-03-03 16:57:28.968399+00:00	000

CAPTURA 11.2: Terminal mostrando [x] Enviado mensaje: cancelar pago con ID=12.

```
(venv) e462971@6B-12-18-12:~/Descargas/P2_SI2/si2_alumnos-main/P1-rpc-frontend$ python cliente_mom/client_mq.py localhost 15672 12
[x] Enviado mensaje: cancelar pago con ID=12
```

CAPTURA 11.3: Resultado de sudo rabbitmqctl list_queues mostrando la cola pago_cancelacion con mensajes pendientes.

```
si2@si2-ubuntu-vm-1:~$ sudo rabbitmqctl list_queues
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name      messages
pago_cancelacion      3
```

CAPTURA 11.4: Terminal del servidor mostrando la recepción y procesamiento del mensaje: [+] Pago 12 cancelado correctamente (codigoRespuesta='111').

```
Last login: Tue Mar  3 18:05:40 2026 from 10.0.2.2
si2@si2-ubuntu-vm-2:~$ cd ~/P1-rpc-backend
python visaAppRPCBackend/server mq.py 10.0.2.2 15672
[*] Esperando mensajes de cancelacion. Pulsa CTRL+C para salir.
[x] Recibido mensaje: cancelar pago con ID=10
[+] Pago 10 cancelado correctamente (codigoRespuesta='111')
[x] Recibido mensaje: cancelar pago con ID=12
[+] Pago 12 cancelado correctamente (codigoRespuesta='111')
[x] Recibido mensaje: cancelar pago con ID=12
[+] Pago 12 cancelado correctamente (codigoRespuesta='111')
```

CAPTURA 11.5: Tabla de pagos del frontend mostrando el pago con codigoRespuesta = 111 (cancelado).

← → ↻ 🏠 ⓘ 127.0.0.1:8001/visaAppRPCFrontend/testbd/getpagos/ ☆ 📁 | A ⋮

🗑️ | 🌐 Escuela Politéc... 🌐 Laboratorio Vi... 🟢 Moodle UAM |...

Pagos Registrados (visaSite)

id	IdComercio	IdTransaccion	Importe	Marca Tiempo	Codigo Respuesta
10	CANCEL001	TRCANCEL01	100.0	2026-03-03 16:57:28.968399+00:00	111

CAPTURA 11.6: Resultado de `sudo rabbitmqctl list_queues` mostrando la cola `pago_cancelacion` con 0 mensajes.

```
si2@si2-ubuntu-vm-1:~$ sudo rabbitmqctl list_queues
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name      messages
pago_cancelacion    0
```


Conclusión

La realización de esta práctica ha permitido comprender de forma práctica los dos paradigmas fundamentales de comunicación en sistemas distribuidos. Por un lado, RPC ofrece una comunicación síncrona y transparente, donde el frontend invoca funciones remotas como si fueran locales gracias al patrón proxy implementado con ServerProxy. Por otro lado, las colas de mensajes proporcionan una comunicación asíncrona y desacoplada, donde el cliente deposita mensajes sin necesidad de que el servidor esté activo en ese momento, lo que mejora la tolerancia a fallos y la flexibilidad del sistema.

La arquitectura final desplegada demuestra cómo una aplicación monolítica puede dividirse en componentes distribuidos que colaboran a través de la red, manteniendo la misma funcionalidad para el usuario. Conceptos como la serialización de datos con `model_to_dict`, el binding estático del servidor RPC y la persistencia de mensajes en RabbitMQ han sido esenciales para lograr una implementación correcta y robusta del sistema.